



SWEN 221

A surreal, dreamlike landscape. In the foreground, a person stands on a dark, rocky outcrop, looking out over a vast, colorful sky. The sky is filled with large, billowing clouds in shades of pink, purple, and blue. A bright, glowing horizon line suggests a sunrise or sunset. In the upper right, a comet streaks across the sky, leaving a trail of light. The overall atmosphere is ethereal and otherworldly.

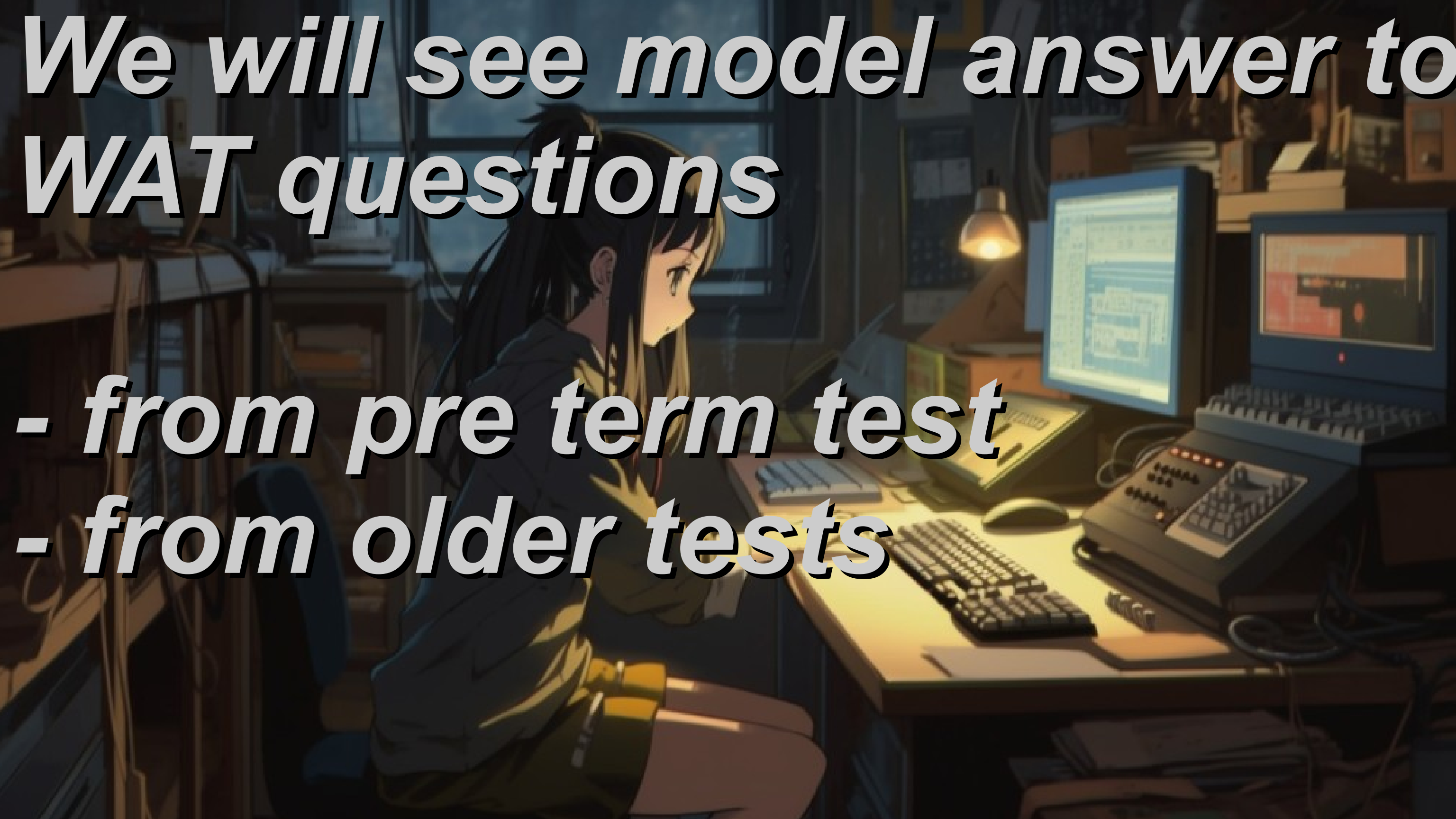
Marco Servetto
CO258

www.youtube.com/MarcoServetto



***Preparing
for
Term
Test 2***

(On 4th May)

An anime-style illustration of a person with long dark hair, seen from the side, sitting at a desk in a dimly lit room at night. The desk is cluttered with two computer monitors, a keyboard, a mouse, and a multi-line office phone. A desk lamp provides a warm glow. The background shows shelves with books and other items, suggesting a workspace or library.

***We will see model answer to
WAT questions***

- from pre term test***
- from older tests***

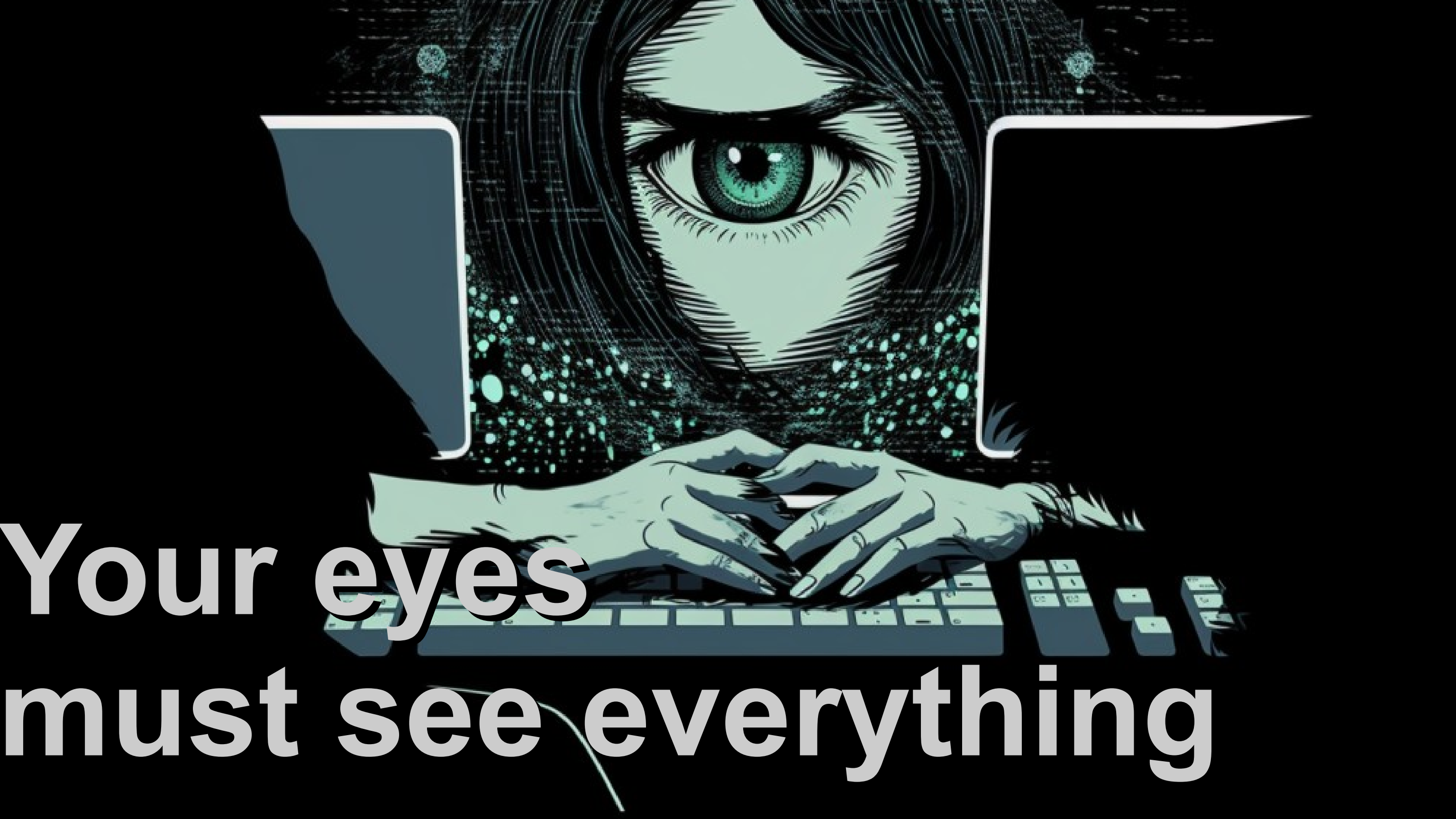
```

class Exercise{//QUESTION 1
    public static String longestString(List<String> ss){ [???] }
    public static void main(String[] arg){
        List<String>ss1 = List.of("aa","a","", "abc");
        assert longestString(ss1).equals("abc"):longestString(ss1);
        List<String>ss2 = List.of("aa","a","abcd","ac");
        assert longestString(ss2).equals("abcd"):longestString(ss2);
        try{ longestString(List.of()); assert false; }
        catch(IllegalArgumentException iae){}
    }
}

class Exercise{//QUESTION 2
    public static int longestStringIndex(List<String> ss){ [???] }
    public static void main(String[] arg){
        List<String>ss1 = List.of("aa","a","", "abc");
        assert longestStringIndex(ss1) == 3 : longestStringIndex(ss1);
        List<String>ss2 = List.of("aa","a","abcd","ac");
        assert longestStringIndex(ss2) == 2 : longestStringIndex(ss2);
    }
}

```

- This is an example of two questions that are sort of similar, and you can have an advantage if you try to understand them together.



Your eyes
must see everything

```

class Exercise{//QUESTION 1
    public static String longestString(List<String> ss){ [???] }
    public static void main(String[] arg){
        List<String>ss1 = List.of("aa", "a", "", "abc");
        assert longestString(ss1).equals("abc"):longestString(ss1);
        List<String>ss2 = List.of("aa", "a", "abcd", "ac");
        assert longestString(ss2).equals("abcd"):longestString(ss2);
        try{ longestString(List.of()); assert false; }
        catch(IllegalArgumentException iae){}
    }
}

class Exercise{//QUESTION 2
    public static int longestStringIndex(List<String> ss){ [???] }
    public static void main(String[] arg){
        List<String>ss1 = List.of("aa", "a", "", "abc");
        assert longestStringIndex(ss1) == 3 : longestStringIndex(ss1);
        List<String>ss2 = List.of("aa", "a", "abcd", "ac");
        assert longestStringIndex(ss2) == 2 : longestStringIndex(ss2);
    }
}

```

- Question 3 has 3 tests in the main, and one of the 3 is using a try-catch..
On the other side, Question 4 asks us to give the '**index**' of the longest string.
- Let's start

//QUESTION 1

```
class Exercise{
    public static String longestString(List<String> ss){
        [???]
    }
    public static void main(String[] arg){
        List<String>ss1 = List.of("aa", "a", "", "abc");
        → assert longestString(ss1).equals("abc"):LongestString(ss1);
        List<String>ss2 = List.of("aa", "a", "abcd", "ac");
        → assert longestString(ss2).equals("abcd"):LongestString(ss2);
        → try{ longestString(List.of()); assert false; }
        catch(IllegalArgumentException iae){}
    }
}
```

- This question asks you to write a method that does a thing.
The main tests such a method.
This is a very common kind of question.
Variants of this question will be in (some) of the term tests.

//QUESTION 1

```
class Exercise{
    public static String longestString(List<String> ss){
        [??]
    }
    public static void main(String[] arg){
        List<String>ss1 = List.of("aa", "a", "", "abc");
        → assert longestString(ss1).equals("abc"):LongestString(ss1);
        List<String>ss2 = List.of("aa", "a", "abcd", "ac");
        assert longestString(ss2).equals("abcd"):LongestString(ss2);
        try{ longestString(List.of()); assert false; }
        catch(IllegalArgumentException iae){}
    }
}
```

- This first test checks that calling 'longestString(ss1)' produces "abc". If the equals fails, we print the result of 'longestString(ss1)'
- When the main reaches "assert somethingFalse: a message" we can see that message as an error!

//QUESTION 1

```
class Exercise{
    public static String longestString(List<String> ss){
        [??]
        ➡ assert ss.size() == 2 : "hi!";

    }
    public static void main(String[] arg){
        List<String>ss1 = List.of("aa", "a", "", "abc");
        assert longestString(ss1).equals("abc"):LongestString(ss1);
        List<String>ss2 = List.of("aa", "a", "abcd", "ac");
        assert longestString(ss2).equals("abcd"):LongestString(ss2);
        try{ longestString(List.of()); assert false; }
        catch(IllegalArgumentException iae){}
    }
}
```

- When the main reaches “assert somethingFalse: a message” we can see that message as an error!
- So... if we write “assert myCondition : myMessage”;
We can actually ‘print out’ the state of what is happening, to help us debugging!!

A close-up, low-angle shot of Optimus Prime's head and upper torso. He is looking directly at the viewer with a serious expression, his blue eyes glowing. His right hand is raised, pointing his index finger forward. The background is a dark, hazy cityscape at night, with some lights visible in the distance. The overall tone is dramatic and heroic.

**Command
your code to help you**

//QUESTION 1

```
class Exercise{
    public static String longestString(List<String> ss){
        [???]
    }
    public static void main(String[] arg){
        List<String>ss1 = List.of("aa","a","", "abc");
        assert longestString(ss1).equals("abc"):longestString(ss1);
        List<String>ss2 = List.of("aa","a","abcd","ac");
        → assert longestString(ss2).equals("abcd"):longestString(ss2);
        try{ longestString(List.of()); assert false; }
        catch(IllegalArgumentException iae){}
    }
}
```

- The second tests checks that, for example, we are not simply returning the last element all of the times.

//QUESTION 1

```
class Exercise{
    public static String longestString(List<String> ss){
        [???]
    }
    public static void main(String[] arg){
        List<String>ss1 = List.of("aa","a","", "abc");
        assert longestString(ss1).equals("abc"):LongestString(ss1);
        List<String>ss2 = List.of("aa","a","abcd","ac");
        assert longestString(ss2).equals("abcd"):LongestString(ss2);
        → try{ longestString(List.of()); assert false; }
           catch(IllegalArgumentException iae){}
    }
}
```

- The third test checks what happens where an empty list “List.of()” is passed to our ‘longestString’ method.
- We have now seen the try-catch in all the details, so we can fully understand what is going on: Let’s think about it together!

//QUESTION 1

```
class Exercise{  
    public static String longestString(List<String> ss){  
        ???  
    }  
    public static void main(String[] arg){...}  
}
```

- So, how do we approach this question?
- From the name of the method and our understanding of the 3 tests, we can understand the expected behavior of the code.
- When working with lists, we often need to use a (for) loop.
- General pattern: Start with a candidate result and improve it step by step while looking at the elements one at a time.

//QUESTION 1

```
class Exercise{  
    public static String longestString(List<String> ss){  
        String candidate = ???;  
        for (var s:ss){  
            ???  
        }  
        return candidate;  
    }  
    public static void main(String[] arg){...}  
}
```

- Start with a **candidate** result and improve it step by step while looking at the elements one at a time.
- Two questions:
 - What is the initial value for ‘candidate’
 - What do we do in the **for**? In the for we have two concepts: ‘candidate’ and ‘s’

```
//QUESTION 1
class Exercise{
    public static String longestString(List<String> ss){
        String candidate= ss.get(0);
        for (var s:ss){
            ???
        }
        return candidate;
    }
    public static void main(String[] arg){...}
}
```

- Start with a **candidate** result and improve it step by step while looking at the elements one at a time.
- Two questions:
 - What is the initial value for ‘candidate’
→ the first element of the list? An empty string? Null?
 - What do we do in the **for**? In the for we have two concepts: ‘candidate’ and ‘s’

```
//QUESTION 1
class Exercise{
    public static String longestString(List<String> ss){
        String candidate= ss.get(0);
        for (var s:ss){
            if (???) { candidate = s; }
        }
        return candidate;
    }
    public static void main(String[] arg){...}
}
```

- What do we do in the **for**? In the for we have two concepts: 'candidate' and 's'
- We need to improve 'candidate' by using 's'

```
//QUESTION 1
class Exercise{
    public static String longestString(List<String> ss){
        String candidate= ss.get(0);
        for (var s:ss){
            if (s.length() > candidate.length()){ candidate = s; }
        }
        return candidate;
    }
    public static void main(String[] arg){...}
}
```

- What do we do in the **for**? In the for we have two concepts: 'candidate' and 's'
- We need to improve 'candidate' by using 's'

//QUESTION 1

```
class Exercise{
    public static String longestString(List<String> ss){

        String candidate= ss.get(0);
        for (var s:ss){
            if (s.length() > candidate.length()){ candidate = s; }
        }
        return candidate;
    }
    public static void main(String[] arg){
        ....
        try{ longestString(List.of()); assert false; }
        catch (IllegalArgumentException iae){}
    }
}
```

- This code passes the first two tests but fails on the third one.
- That is, what happens when ss is empty?
- What should happen instead?

//QUESTION 1

```
class Exercise{
    public static String longestString(List<String> ss){
        if (ss.isEmpty()){ throw new IllegalArgumentException(); }
        String candidate= ss.get(0);
        for (var s:ss){
            if (s.length() > candidate.length()){ candidate = s; }
        }
        return candidate;
    }
    public static void main(String[] arg){
        ....
        try{ longestString(List.of()); assert false; }
        catch(IllegalArgumentException iae){}
    }
}
```

- Remember: The empty list is a list. The empty list is a list. The empty list is a list.

//QUESTION 1

```
class Exercise{
    public static String longestString(List<String> ss){
        if (ss.isEmpty()){ throw new IllegalArgumentException(); }
        String candidate= ss.get(0);
        for(int i= 1; i < ss.size(); i += 1){
            var s= ss.get(i);
            if (s.length() > candidate.length()){ candidate = s; }
        }
        return candidate;
    }
    public static void main(String[] arg){...}
}
```

- Alternative solution: by using indexes we can avoid checking what is longest between the 'first' and the 'first' element of the list.

//QUESTION 1

```
class Exercise{
    public static String longestString(List<String> ss){
        return ss.stream()
            .max( (s1,s2)->Integer.compare(s1.length(),s2.length()) )
            .orElseThrow( IllegalArgumentException::new );
    }
    public static void main(String[] arg){...}
}
```

- Alternative solution using streams.
Questions in Term Test 2,3,4 may force you to use streams.
- The streams API pushes us to consider the empty list case.

```
class Exercise{//QUESTION 2
    public static int longestStringIndex(List<String> ss){
        ???

    }

    public static void main(String[] arg){ ... }
}
```

- Here we want an 'int' index out
- We need to keep track of the indexes, but '`for (var s:ss) { .. }`' does not directly gives us indexes...

```

class Exercise{//QUESTION 2
    public static int longestStringIndex(List<String> ss){
        if(ss.isEmpty()){ throw new IllegalArgumentException(); }
        int candidate= 0;
        for (int i= 1; i < ss.size(); i += 1){
            var s= ss.get(i);
            var better= s.length() > ss.get(candidate).length();
            if (better){ candidate = i; }
        }
        return candidate;
    }
}

public static void main(String[] arg){ ... }
}

```

- So we can use the more primitive index based ‘for’.
- Same idea as before: candidate + ‘for’ with a tests to improve candidate.
- Feel free to declare more (or less) local variables if you find it more readable in that way.
- For example I’m using a local variable for the result of the ‘if condition’.

```

class Exercise{//QUESTION 2
    public static int longestStringIndex(List<String> ss){
        return IntStream.range(0,ss.size()).boxed()//boxed gives Stream<Integer>
            .max((i1,i2)
                ->Integer.compare(ss.get(i1).length(),ss.get(i2).length()))
            .orElseThrow(    IllegalArgumentException::new    );
    }

    public static void main(String[] arg){ ... }
}

```

- With streams: as you can see, streams do work even if we need indexes.

```

class Exercise{//QUESTION 3
    public static String prettyPrint(List<String> names, List<String> surnames){
        if (names.size() != surnames.size()){ throw new IllegalArgumentException(); }
        return IntStream.range(0,names.size())
            .mapToObj( i->names.get(i) + " " + surnames.get(i) )
            .collect( Collectors.joining("\n") );
    }
    public static void main(String[] arg){
        String bs= "["+ prettyPrint(List.of("Bob", "Bart", "Billy"),
                                         List.of("McKain","Brown","Jolly"))+"]";
        assert bs.equals("[Bob McKain\nBart Brown\nBilly Jolly]"):bs;
    }
}

```

- Here we show how to iterate with streams on two lists at the same time

Complete the code

```
import java.util.List;
import java.util.Set;
import java.util.stream.Collectors;
import java.util.stream.IntStream;

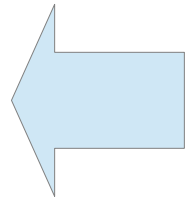
class Exercise {//QUESTION 4
    public static void main(String[] args) {
        MyClass mc= new MyClass();
        assert mc.getValue() == 10: mc.toString();
        assert mc.add(3) == 13: mc.toString();
        assert mc.add(-6) == 7: mc.toString();
        MyClass mc2= new MyClass();
        assert mc2.add(-6) == 4: mc2.toString();
        assert mc.getValue() == 7: mc.toString();
    }
}
```

[???

```
import java.util.List;
import java.util.Set;
import java.util.stream.Collectors;
import java.util.stream.IntStream;

class Exercise {//QUESTION 4
    public static void main(String[] args) {
        MyClass mc= new MyClass();
        assert mc.getValue() == 10: mc.toString();
        assert mc.add(3) == 13: mc.toString();
        assert mc.add(-6) == 7: mc.toString();
        MyClass mc2= new MyClass();
        assert mc2.add(-6) == 4: mc2.toString();
        assert mc.getValue() == 7: mc.toString();
    }
}
```

[???



The imports at the top are just a distraction.

I will insert distractions in the questions.

You need to grow the skill to navigate around those. Real world situations are going to be full of code that is not directly relevant to solve the problem at hand.

```
import java.util.List;
import java.util.Set;
import java.util.stream.Collectors;
import java.util.stream.IntStream;
```

As you can see 'MyClass'
is used in the main but is not
declared anywhere.

```
class Exercise { //QUESTION 4
    public static void main(String[] args) {
        MyClass mc= new MyClass();
        assert mc.getValue() == 10: mc.toString();
        assert mc.add(3) == 13: mc.toString();
        assert mc.add(-6) == 7: mc.toString();
        MyClass mc2= new MyClass();
        assert mc2.add(-6) == 4: mc2.toString();
        assert mc.getValue() == 7: mc.toString();
    }
}
```

[???

```
import java.util.List;
import java.util.Set;
import java.util.stream.Collectors;
import java.util.stream.IntStream;
```

We use 'getValue' and 'add'

They both seams to return an int

```
class Exercise {//QUESTION 4
    public static void main(String[] args) {
        MyClass mc= new MyClass();
         assert mc.getValue() == 10: mc.toString();
         assert mc.add(3) == 13: mc.toString();
        assert mc.add(-6) == 7: mc.toString();
        MyClass mc2= new MyClass();
        assert mc2.add(-6) == 4: mc2.toString();
        assert mc.getValue() == 7: mc.toString();
    }
}
```

[???

```

class Exercise { //QUESTION 4
    public static void main(String[] args) {
        MyClass mc= new MyClass();
        assert mc.getValue() == 10: mc.toString();
        assert mc.add(3) == 13: mc.toString();
        assert mc.add(-6) == 7: mc.toString();
        MyClass mc2= new MyClass();
        assert mc2.add(-6) == 4: mc2.toString();
        assert mc.getValue() == 7: mc.toString();
    }
}

```

```

class MyClass {
    private int value= ???;
    public int getValue(){ ???; }
    public int add(int x){ ???; }
}

```

Skeleton of the solution

You could logically divide questions like this in 2 parts:

1- make it compile

2- make it run without errors

```

class Exercise { //QUESTION 4
    public static void main(String[] args) {
        MyClass mc= new MyClass();
        assert mc.getValue() == 10: mc.toString();
        assert mc.add(3) == 13: mc.toString();
        assert mc.add(-6) == 7: mc.toString();
        MyClass mc2= new MyClass();
        assert mc2.add(-6) == 4: mc2.toString();
        assert mc.getValue() == 7: mc.toString();
    }
}

```

We can see that the value starts at 10 and that the 'add' method does add the parameter to value.

```

class MyClass {
    private int value= 10;
    public int getValue(){ return value; }
    public int add(int x){ value += x; return value; }
}

```

```
class Exercise {//QUESTION 5
    public static void main(String[] args) {
        A a= new A();
        assert a.b().zero() == 0: a.toString();
        assert a.b().one() == 1: a.toString();
    }
}
```

???

This question is harder since the 'b' method is returning some object with other methods, that we also have to define.

```
class Exercise {//QUESTION 5
    public static void main(String[] args) {
        A a= new A();
        assert a.b().zero() == 0: a.toString();
        assert a.b().one() == 1: a.toString();
    }
}
```

```
class A{
    ??? b(){ return ???; }
}
```

```
class Exercise {//QUESTION 5
    public static void main(String[] args) {
        A a= new A();
        assert a.b().zero() == 0: a.toString();
        assert a.b().one() == 1: a.toString();
    }
}
```

```
class A{
    B b(){ return new B(); }
}
class B{
    int zero(){ return 0; }
    int one(){ return 1;}
}
```

This is one solution

```
class Exercise {//QUESTION 5
    public static void main(String[] args) {
        A a= new A();
        assert a.b().zero() == 0: a.toString();
        assert a.b().one() == 1: a.toString();
    }
}
```

```
class A{
    A b(){ return this; }
    int zero(){ return 0; }
    int one(){ return 1;}
}
```

This is another,
more compact solution.

Here making it compile was the
harder task. Implementing the two
zero and one methods is quite trivial.

```

class Exercise { //QUESTION 6
    public static void main(String[] args) {
        A a= new A();
        assert a.b().zero() == 0: a.toString();
        assert a.b().one() == 1: a.toString();
        assert a.b().zero() == 2: a.toString();
        @SuppressWarnings("unused")
        A aa= new A();
        assert a.b().zero() == 0: a.toString();
        //Yes, it is not a typo.
        //We do mean 'a', not 'aa'.
    }
}

```

This is an interesting variant,
where the method behavior does
NOT follow from the method name!

The hard task is to figure out how to
get the needed behavior!

```

class Exercise { //QUESTION 6
    public static void main(String[] args) {
        A a= new A();
        assert a.b().zero() == 0: a.toString();
        assert a.b().one() == 1: a.toString();
        assert a.b().zero() == 2: a.toString();
        @SuppressWarnings("unused")
        A aa= new A();
        assert a.b().zero() == 0: a.toString();
        //Yes, it is not a typo.
        //We do mean 'a', not 'aa'.
    }
}

```

```

class A {
    { B.count = -2; }
    B b() { return new B(); }
}
class B{
    static int count= -2;
    int zero() { count += 2; return count; }
    int one() { return 1; }
}

```

The idea: creating a new A updates a static counter!

```

class Exercise { //QUESTION 6
    public static void main(String[] args) {
        A a= new A();
        assert a.b().zero() == 0: a.toString();
        assert a.b().one() == 1: a.toString();
        assert a.b().zero() == 2: a.toString();
        @SuppressWarnings("unused")
        A aa= new A();
        assert a.b().zero() == 0: a.toString();
        //Yes, it is not a typo.
        //We do mean 'a', not 'aa'.
    }
}

```

Other possible solution

```

class A {
    A(){ count = 0; }
    A b() { return this; }
    static int count;
    int zero() { return count++; }
    int one() { return count++; }
}

```

```
import java.util.List;
import java.util.Set;
import java.util.stream.Collectors;
import java.util.stream.IntStream;
```

The method here does
follow the intuition from the
name

```
class Exercise{ //QUESTION 7
    public static List<String> removeDups(List<String> ss){
        [???]
    }
    public static void main(String[] args) {
        var s1 = removeDups(List.of("Hi", "Hello", "Hi"));
        var s2 = removeDups(List.of("a", "b", "c", "d", "b", "d", "a", "z"));
        var s3 = removeDups(List.of("a"));
        var s4 = removeDups(List.of());
        assert s1.equals(List.of("Hi", "Hello"));
        assert s2.equals(List.of("a", "b", "c", "d", "z"));
        assert s3.equals(List.of("a"));
        assert s4.equals(List.of());
    }
}
```

It creates a new lists that
looks like the one in input,
but without the duplicated
strings.

Note: we do not import ArrayList. You can still use it!

```
import java.util.List;
import java.util.Set;
import java.util.stream.Collectors;
import java.util.stream.IntStream;

class Exercise{ //QUESTION 7
    public static List<String> removeDups(List<String> ss){
        return new java.util.ArrayList<String>(
            new java.util.HashSet<String>(ss));
    }
    public static void main(String[] args) {...}
}
```

The simplest solution:
list → set → list

Most collections have a constructor that takes another collection.

This solution does pass the tests, but it could have not.

Can you tell me why?

```
import java.util.List;
import java.util.Set;
import java.util.stream.Collectors;
import java.util.stream.IntStream;
```

```
class Exercise{ //QUESTION 7
    public static List<String> removeDups(List<String> ss){
        return new java.util.ArrayList<String>(
            new java.util.LinkedHashSet<String>(ss));
    }
    public static void main(String[] args) {...}
}
```

The simplest solution:
list → set → list

Learning moment:

LinkedHashSet/LinkedHashMap

LinkedHashSet ensures
that the order is preserved.

They work like HashSet/HashMap, but iteration order is the same of insertion order instead of being undefined.

Speed of LinkedHashSet/LinkedHashMap is near the same, space consumption is a little higher.

```

import java.util.List;
import java.util.Set;
import java.util.stream.Collectors;
import java.util.stream.IntStream;

class Exercise{ //QUESTION 7
    public static List<String>
removeDups(List<String> ss){
    var seen= new java.util.HashSet<String>();
    var res= new java.util.ArrayList<String>();
    for(var s:ss){
        if(seen.add(s)){ res.add(s); }
    }
    return res;
}
    public static void main(String[] args) {...}
}

```

This is a viable but time consuming solution:

Create a set of seen elements and a list result. if the addition succeeded, then s must have been the first occurrence of that string.

This required remembering
 -the existence of sets
 -the method add.

```
import java.util.List;
import java.util.Set;
import java.util.stream.Collectors;
import java.util.stream.IntStream;

class Exercise{ //QUESTION 7
    public static List<String>
removeDups(List<String> ss){
    return ss.stream().distinct().toList();
}
    public static void main(String[] args) {...}
}
```

This is the
professional solution:
Use streams

So, what should I know for the test?

- Same term test document for all the tests, so look to the one for term test 1.
- You should read and **try to grasp** the full text from the documentation of the methods listed in that document.
- List here <https://docs.oracle.com/en/java/javase/26/docs/api/java.base/java/util/List.html>
- Navigate from there for the other methods
- You should also read the documentation for the constructors for ArrayList, HashSet, HashMap.

But.. what do I need to memorize?

- You need to get a very good grasp of the names and behavior for the most important methods of those types (listed in the term test document)
- Overall, 'memorize' is a bad term.

Register for your preferred venue (lecture theater or Lab)

- Use myAllocator, to chose where you want to attend!
- You will need to allocate yourself again (some allocations have been lost)
- Deadline To allocate yourself: Tomorrow Friday 01 May
- As usual, if you have any special situation and/or you can not attend on the day of the test, write me ASAP



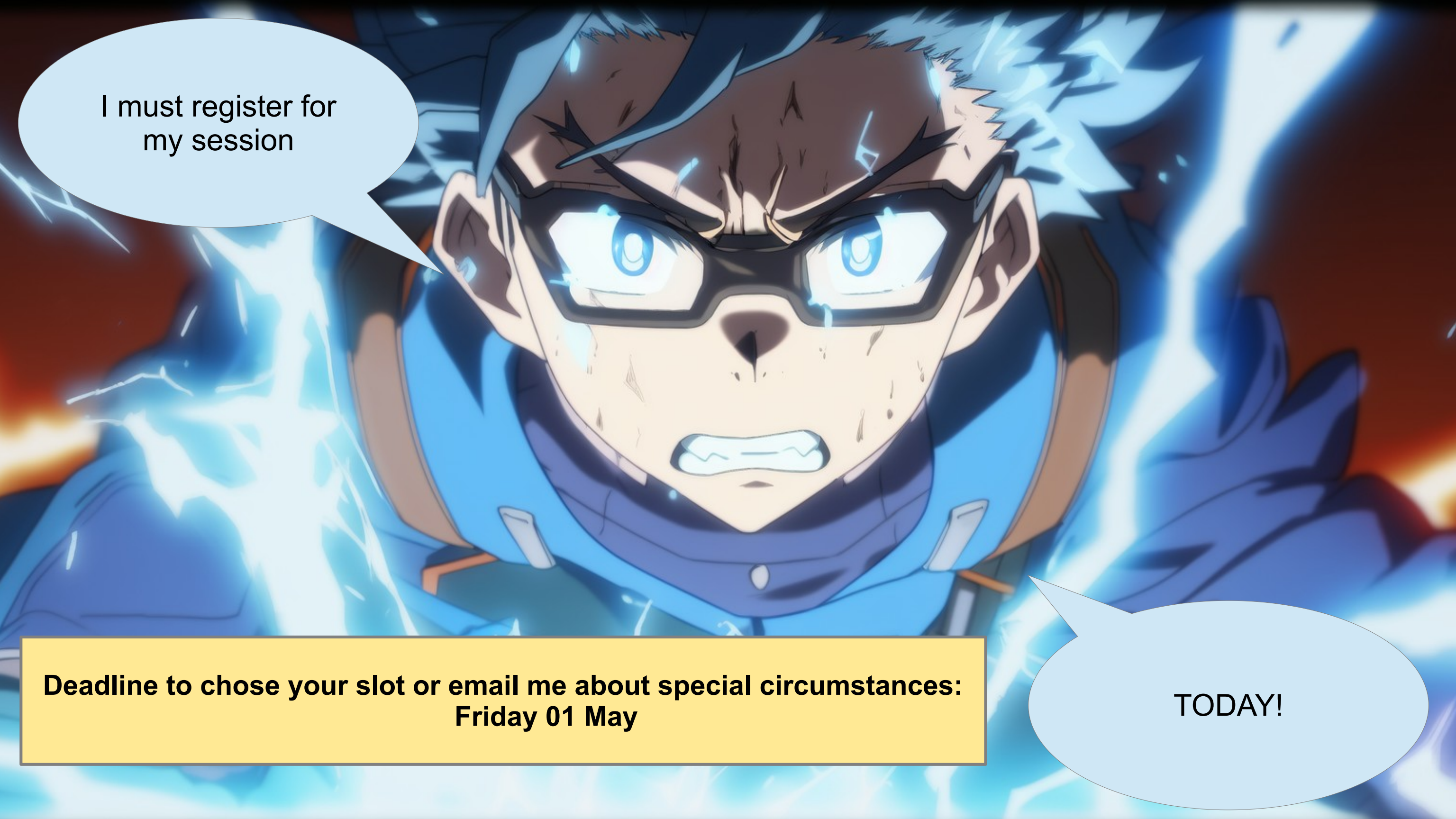
I must register for
my session





I must register for
my session

TODAY!



I must register for
my session

**Deadline to chose your slot or email me about special circumstances:
Friday 01 May**

TODAY!

What if I can get covid (or another disaster) at the last minute?

- Write me an email anyway as soon as possible.
- I will see what I can do (and email you about it).

Instructions and rules

- Wait outside of the room until we call you in. Get your student ID ready!
- When invited in, sit on one of the suggested places. We will start filling the rooms from the top.
- When we provide you with the link, open the link.
- You must keep open ONLY the browser and ONLY with the provided link.
- However, you can have as many tabs open as you want, if they all are with the provided link.

- We will 'check in' and 'check out' you using the students ID. Make sure we do it!
- If you do not have access to your student ID, try to fix the issue right now!
- If you somehow still can not provide the student id, bring some other ID document and we will discuss what to do at that moment.

- If you are booked to be in the lecture theater and your laptop breaks, just go to the lab session.
- If your laptop breaks in the middle of the test, relax, and ask for help, we will move you to the lab session.

Suggested Strategy

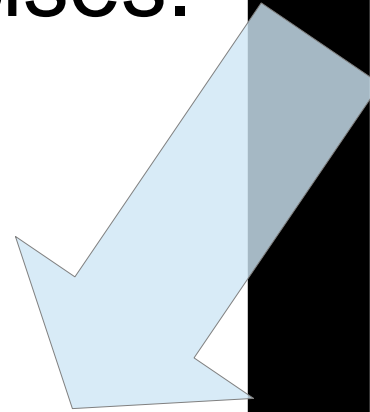
- When we provide you with the link, open the link, then open a new tab with each question.
- Read all the code from all the questions, this should take about 10 minutes.
- Then, try to solve the questions in order.
- If you get stuck for more than 10 minutes, try another question.
Remember: you can just get back to that question later.
- When you are stuck on all your remaining questions, try to read all the code of all the questions again (including the ones you already completed).
- Do not lose hope if you are stuck. Relax, and think about the problem from other points of view.
- Use the back key of the browser to 'recover' your last incorrect answer.
- Try to use as many attempts as possible, many successful students can make a new attempt every 20-40 seconds (and most of those attempts will fail, of course).
- After many failed attempts, stop and think. Then get another round of fast attempts.
- If you manage, try to divide each question in 2 parts:
(1) making the code compile, (2) making the run time tests pass

Lab Week 7 is designed to help you with Term Test 2/3

- The best preparation for term test 2/3 is to try to do lab week 7 with care
 - Have you noticed that there was a lab handout (as for all the other labs)
- After doing the lab week 7 while following the handout, try to do it again but in 'exam conditions':
 - without access to internet and the course material

Please attempt Part 3!

- Personal exercises, and a guide on how to do personal exercises.
- Let's try together?



Note that the question suggestions below are not intended to be an exhaustive list of all questions we may ask, but they are just to support you in your preparation.

For example: as you have noticed, we have seen many questions of the form “find the tallest person living in a city (from a list)”
“find the nearest person to a point (from a map)”

I'm sure you can see that there is a pattern here, and that there could be many more questions on the same pattern, for example: “find the tallest flower (from a set)”, “find the largest biscuit (from a list of biscuit packs)”, “find the name of the youngest person (from a set)”.

All of those questions could be asked considering the empty case, requiring streams, requiring recursion, or using some other conditions (like selecting only the chocolate biscuits).

Other kinds of questions may require to go from lists to list; for example go from a list of Persons to the list of all their names, removing duplicates.

Again, you could do it with streams, with recursion or with a for statement.

Part 3: Personal exercise

If you have answered all the questions above, you have now completed lab 5.

To better prepare for Term Test 2, you may want to do some personal exercises.

The text in this section is going to help you to create more questions and exercises that you can do on your own. Note that the question suggestions below are not intended to be an exhaustive list of all the possible kinds of questions we may ask, but they are just to support you in your preparation.

For example: as you have noticed, we have seen many questions of the form “find the longest string from a list”
“find the tallest person living in a city (from a list)”
“find the nearest person to a point (from a map)”

I'm sure you can see that there is a pattern here, and that there could be many more questions on the same pattern, for example: “find the tallest flower (from a set)”, “find the largest biscuit (from a list of biscuit packs)”, “find the name of the youngest person (from a set)”.

All of those questions could be asked considering the empty case, requiring streams, requiring recursion, or using some other conditions (like selecting only the chocolate biscuits).

Other kinds of questions may require to go from lists to list; for example go from a list of Persons to the list of all their names, removing duplicates.

Again, you could do it with streams, with recursion or with a for statement.

The question about Kangaroos can also be easily adapted: we can have Cars that need to have a license plate following certain rules, min-max weight and other constraints.

We could have Houses that need to have certain constraints about the number of windows with respect to the number of residents.

For another example, can you imagine some constraints for a WashingMachine?

In this Lab, except for the single question about exceptions from Term Test 1, we did not explore any questions requiring to declare your own exception types. Those kind of exceptions could be present in Term Test 2-3-4-5. Can you try to imagine how such a question may look like?

To do personal exercises, you can make new questions following those ideas and try to write your own tests and solutions.

How this looks in practice?

- Open eclipse, make a new project called 'PersonalExercises'
- make a new package for each question
- cut-paste one of the questions from the tool
- answer the question and run the main to see that it is working.
- check that assertions are enabled by adding an “assert false;” at the start of the main. You should see that running the main again the question now fails.
- Change the question to something of your own creation using the examples proposed above.
- Solve it.
- Make a new package and make a new question to continue your personal exercises.

Steps:

- 1 Make a new package (tallestFlower)
- 2 cut paste an exercise
- Try to run, see that it pass
- Add `assert false;` at the start of the method
- See that now it is failing (otherwise learn how to enable assertions)
- Then change the exercise into our new exercise!

```
package tallestFlower;

import java.util.List;

record Person( String name, int height, String city ){}

public class Exercise {
    public static Person tallestPersonIn(List<Person> ps, String city){
        Person candidate=null;
        for(var p:ps){
            if(!p.city().equals(city)){ continue; }
            if(candidate==null || candidate.height()<p.height()){ candidate=p; }
        }
        if(candidate==null){ throw new IllegalArgumentException(); }
        return candidate;
    }
    static Person p(String name, int height, String city){
        return new Person(name,height,city);
    }
    final static List<Person> ps1 = List.of(
        p("Bob",178,"Wellington"),p("Alice",167,"Wellington"),
        p("Adam",201,"Auckland"),p("Alice",162,"Auckland"),
        p("Gianni",150,"Genoa"),p("Maria",160,"Genoa"),
        p("Napoleon",140,"Paris"),p("Rose",190,"Genoa")
    );
    final static List<Person> ps2 = List.of(
        p("Bob",178,"Wellington"),p("Alice",162,"Auckland"),
        p("Gianni",150,"Genoa"),p("Maria",160,"Genoa"),
        p("Napoleon",140,"Paris"),p("Rose",140,"Genoa")
    );
    public static void main(String[] a) {
        assert false;
        assert
tallestPersonIn(ps1,"Wellington").equals(p("Bob",178,"Wellington"));
        assert
tallestPersonIn(ps2,"Wellington").equals(p("Bob",178,"Wellington"));
        assert tallestPersonIn(ps1,"Auckland").equals(p("Adam",201,"Auckland"));
        assert tallestPersonIn(ps2,"Auckland").equals(p("Alice",162,"Auckland"));
        assert tallestPersonIn(ps1,"Genoa").equals(p("Rose",190,"Genoa"));
        assert tallestPersonIn(ps2,"Genoa").equals(p("Maria",160,"Genoa"));
        try{ tallestPersonIn(ps1,"Misspelled"); assert false; }
        catch(IllegalArgumentException iae){}
        try{ tallestPersonIn(List.of(),"Genoa"); assert false; }
        catch(IllegalArgumentException iae){}
    }
}
```

```
package tallestFlower;
```

```
import java.util.List;
```

```
record Person( String name, int height, String city ){}  
  
public class Exercise {  
    public static Person tallestPersonIn(List<Person> ps, String city){  
        [??]  
    }  
    static Person p(String name, int height, String city){  
        return new Person(name,height,city);  
    }  
    public static void main(String[] a) {  
        assert tallestPersonIn(...,"Wellington").equals(p("Bob",178,"Wellington"));  
        ....//more tests here  
        try{ tallestPersonIn(...,"Misspelled"); assert false; }  
        catch(IllegalArgumentException iae){}  
        try{ tallestPersonIn(List.of(),"Genoa"); assert false; }  
        catch(IllegalArgumentException iae){}  
    }  
}
```

- First, we can adapt the data types so that we have flowers and not persons

- Then we write/adapt our tests

- After we can try to solve the new question

```
package tallestFlower;
```

```
import java.util.Set;
```

```
record Flower(int height, String city){}
```

```
public class Exercise {  
    public static Flower tallestFlowerIn(Set<Flower> ps, String city){  
        [??]  
    }
```

```
    static Flower f(int height, String city){  
        return new Flower(height, city);  
    }
```

```
    public static void main(String[] a) {  
        assert tallestFlowerIn(..., "Wellington").equals(f(58, "Wellington"));  
        ....//more tests here  
        try{ tallestFlowerIn(..., "Misspelled"); assert false; }  
        catch(IllegalArgumentException iae){}  
        try{ tallestFlowerIn(List.of(), "Genoa"); assert false; }  
        catch(IllegalArgumentException iae){}  
    }  
}
```

- First, we can adapt the data types so that we have flowers and not persons

- Then we write/adapt our tests

- After we can try to solve the new question

**Here I'm using asserts
in the main to be as
similar as possible to
the term test.**

**Feel free to make your
own Junit test suite
instead,... or both!**

Unit tests



Some example model solutions

```
record Flower(int height, String city){}
...
public static Flower tallestFlowerIn(Set<Flower> fs, String city){
    Flower res= null;
    for (Flower f : fs){
        if (!f.city().equals(city)) { continue; }
        if (res == null || res.height() < f.height()) { res = f; }
    }
    if (res == null) { throw new IllegalArgumentException(); }
    return res;
}
```

- Old style with for/if statements
 - Declare a tentative result
 - Make a for where you improve the tentative result while iterating
 - At the end, if the tentative result is still 'bad' throw the right error
 - Return the tentative result

Some example model solutions

```
record Flower(int height, String city){}
...
public static Flower tallestFlowerIn(Set<Flower> fs, String city){
    return fs.stream()
        .filter(f-> f.city().equals(city))
        .reduce((f1, f2)-> f1.height() > f2.height() ? f1 : f2)
        .orElseThrow(IllegalArgumentException::new);
}
```

- New style with streams - reduce:
 - Filter the flowers outside of the city
 - Reduce the potentially valid flowers to find the tallest
 - At the end, if the optional is empty throw the right error
 - The whole thing often fits in the expression of a top level return

Some example model solutions

```
record Flower(int height, String city){}
...
public static Flower tallestFlowerIn(Set<Flower> fs, String city){
    return fs.stream()
        .filter(f-> f.city().equals(city))
        .max(Comparator.comparing(Flower::height))// or f->f.height()
        .orElseThrow(IllegalArgumentException::new);
}
```

- New style with streams - max:
 - Filter the flowers outside of the city
 - Make a comparator that knows what to compare about
 - At the end, if the optional is empty throw the right error
 - In the end, pretty similar to before

Different styles

1-----
`return fs.stream()
 .filter(f-> f.city().equals(city))
 .reduce((f1, f2)-> f1.height() > f2.height() ? f1 : f2)
 .orElseThrow(IllegalArgumentException::new);`

2-----
`return fs.stream()
 .filter(f-> f.city().equals(city))
 .max(Comparator.comparing(Flower::height))
 .orElseThrow(IllegalArgumentException::new);`

3-----
`Flower res= null;
for (Flower f : fs){
 if (!f.city().equals(city)) { continue; }
 if (res == null || res.height() < f.height()) { res = f; }
}
if (res == null) { throw new IllegalArgumentException(); }
return res;`



Different styles

Both styles have advantages and disadvantages.

We need to learn to read and write code in both styles



Preparing for the term test

- Redo lab week 7 in exam conditions
- Try to complete all the WAT questions
- Re-read all the slides
- Try to do some personal exercise as described in Lab week 7 part 3



Unexpected events

- You are sitting in the exam room and your laptop breaks
 - You are sitting in the lab and your lab machine breaks
 - You suddenly start feeling very sick
 - You are unable to reach the session you booked on myAllocator
 - You got COVID the night before the term test
-
- Relax and call for help.

We will record the situation and try to handle it at our best to give you a fair shot at sitting those term tests.



More model solutions

```
record Kangaroo(String name, double weight, boolean pouch){  
  Kangaroo{  
    assert name != null && !name.isBlank() && !name.contains("\n");  
    assert weight > 0 && weight < 500;  
    assert !pouch || weight < 300;  
  }  
}
```

Representation invariants! Here we can use asserts to check that all Kangaroos have a valid state.

- Note the syntax `Kangaroo{...}`
- This is called a ‘compact constructor’. We have seen it already a few times!
- Make sure to remember how to use a compact constructor for Term Test 2.

More model solutions

```
static Point addAll(List<Point> ps){  
    if(ps.isEmpty()){ return new Point(0,0); }  
    var rec= addAll(ps.subList(1,ps.size()));  
    return rec.add(ps.get(0));  
}
```

Recursion is a thing you should remember from first year!

- By using subList you can recursively explore a list.
- This is quite efficient, the subList method does not copy the whole list, just makes a wrapper with a start/end indexes.
- There may be a question asking to use recursion in Term test 2

More model solutions

```
class Box<T>{  
    private T f;  
    Box(T f){ this.f= f; }  
    T get(){ return f; }  
    void set(T f){ this.f = f; }  
    public String toString(){ return f.toString(); }  
    public int hashCode(){ return f.hashCode(); }  
    public boolean equals(Object o){  
        if(!(o instanceof Box<?> b)){ return false; }  
        return this.f.equals(b.f);  
    }  
}
```

Generics!

The question BoxOfT asked you to write a transparent box of a generic type T.

- This is how to encode it.
- Make sure to understand this in very detail for Term Test 2

More model solutions

```
class Persons{  
    Map<Person,String> map(){ return Map.of(  
        new Person("Bob",160),"Bob",  
        new Person("Alice",180),"Alice"  
    );}  
}
```

The first novel question of Lab week 7:
Just make a Map with some data in.

- This question is very easy if you understand the concept of unmodifiable collections and are skilled in using maps.
- For Term test 2, make sure to know what is the difference between unmodifiable and modifiable collections (list/map/set), and how to create and manipulate those two kinds of collections.
- That is, what is the difference between
`List.copyOf(es)`, `Collections.unmodifiableList(es)`, `new ArrayList<>(es)`

More model solutions

```
record Person(String name){}
record Point(int x,int y){
    double distance(Point other){
        double xd= x - other.x;
        double yd= y - other.y;
        return Math.sqrt(xd * xd + yd * yd);
    }
}
public static Person nearestPerson(Map<Person,Point> ps, Point target){
    Person candidate= null;
    double distance= Double.MAX_VALUE;
    for (Person p : ps.keySet()){
        double newD= ps.get(p).distance(target);
        if (newD < distance){ distance = newD; candidate = p; }
    }
    if (candidate == null){ throw new IllegalArgumentException(); }
    return candidate;
}
```

Question NearestPerson from Term Test 1

- To iterate on a map, you can either
 - iterate on `.entrySet()` and use `.getKey()` and `.getValue()`
 - Iterate on `.keySet()` and use `.get()` to get the value //slow but simple
 - Use streams on `.entrySet()`

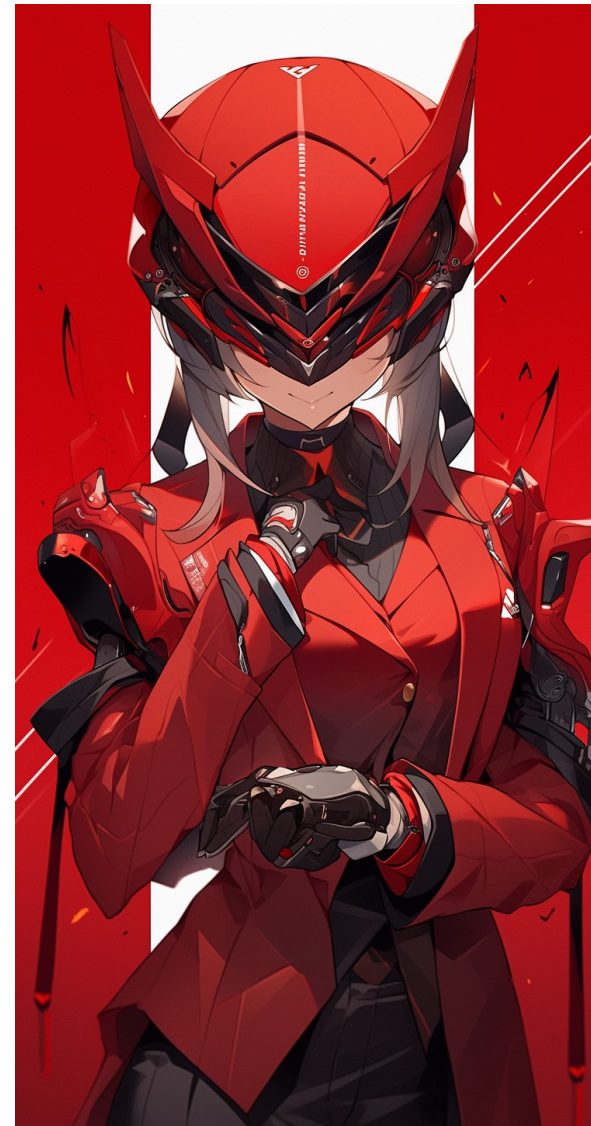


More model solutions

```
record Person(String name){}
record Point(int x,int y){
    double distance(Point other){
        double xd= x - other.x;
        double yd= y - other.y;
        return Math.sqrt(xd * xd + yd * yd);
    }
}
public static Person nearestPerson(Map<Person,Point> ps, Point target){
    return ps.entrySet().stream()
        .min(Comparator.comparing(e->e.getValue().distance(target)))
        .map(e->e.getKey())//map on optional here!
        .orElseThrow(IllegalArgumentException::new);
}
```

Question NearestPerson from Term Test 1 but with streams!

- Do you notice how the streams solutions all look like each other?



Do I need stream and optional for Term Test 2?

- There will be questions using streams, optionals and lambdas as there are in Lab week 7. So, you can still get some marks without Streams/Optionals/Lambdas for Term Test 2, but you will need them to get a good mark.
- More in the details: Term Test 2 is going to cover:
 - All the first year content
 - All the content of SWEN221 up to Lecture 19 included.
- This means that the content at the start of the course is intentionally assessed multiple times, since it is so important.

**Hopefully, this is your
expression after
Terms test 2**

**We can do it,
but we need to take
it seriously!**

